

Coordination & communication

language constructs for concurrent and distributed programming

6 Asynchronous Programming

7 Multi-Threading

8 Message Passing

9 Failure and Recovery

A sequential program and a slow device

```
// k numbers the portions; the bound of portion k is 250000 + 83333 * (k % 3)
for (int k = 1; k <= 8; k++) {
    // a computation: how many primes are there below the bound of portion k?
    String portion = produceNextPortion(k);
    // a slow device: prints the answer as one line of the page
    processPortionTaken(portion);
}
```

[Java ↗](#)

Run

```
line 1: 28665 primes below 333333
line 2: 35131 primes below 416666
line 3: 22044 primes below 250000
line 4: 28665 primes below 333333
line 5: 35131 primes below 416666
line 6: 22044 primes below 250000
line 7: 28665 primes below 333333
line 8: 35131 primes below 416666
220 ms
```

recorded · Java 26

A **portion** is one line of text: the count of primes below a bound, three bounds in turn so that the portions take different times. The **device** is slow: a pause of 5 to 25 ms before the line reaches the page. Eight lines in order, 220 ms, most of it waiting for the device.

Dijkstra's bounded buffer (1965)

What will be true in every run?

k produced k processed

134 ms	174 ms	186 ms	140 ms	166 ms
1	1	1	1	1
2	1	2	1	1
1	2	3	2	2
3	3	1	3	3
4	4	4	4	4
2	2	5	2	5
5	5	2	5	2
3	3	6	3	6
6	6	3	6	3
4	7	7	4	7
7	4	4	7	4
8	5	8	8	5
5	8	5	5	8
6	6	6	6	6
7	7	7	7	7
8	8	8	8	8

```
// producer
```

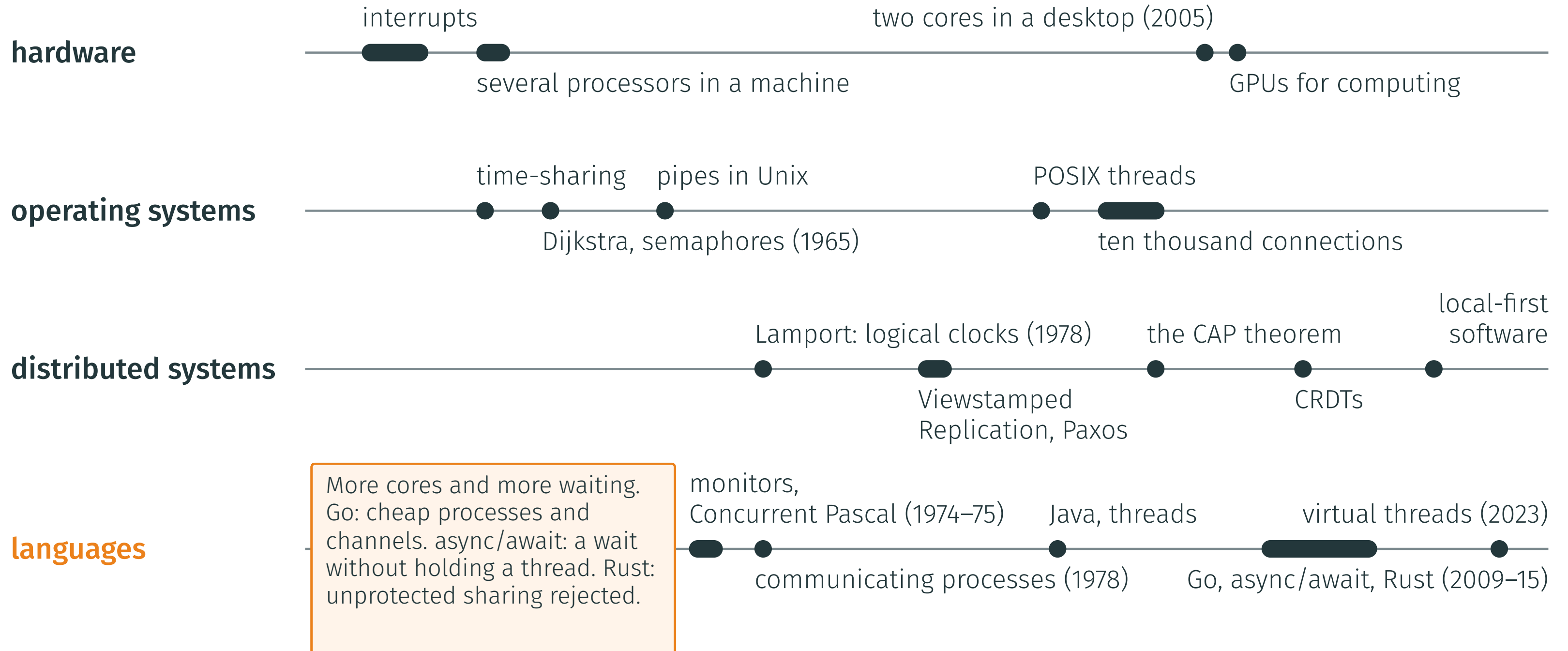
```
for (int k = 1; k <= 8; k++) {  
    String portion = produceNextPortion(k);  
    numberOfEmptyPositions.acquire();    // wait for room  
    bufferManipulation.acquire();        // one at a time on the queue  
    buffer.addLast(portion);  
    bufferManipulation.release();  
    numberOfQueuingPortions.release();  
}
```

```
// consumer
```

```
for (int k = 1; k <= 8; k++) {  
    numberOfQueuingPortions.acquire();    // wait for a portion  
    bufferManipulation.acquire();        // one at a time on the queue  
    String portion = buffer.removeFirst();  
    bufferManipulation.release();  
    numberOfEmptyPositions.release();  
    processPortionTaken(portion);  
}
```

[Java ↗](#)

A short history of concurrent and distributed programming



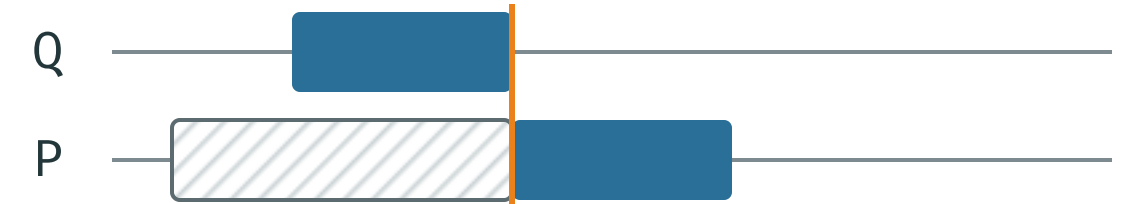
Coordination and Communication

Coordination and communication

Coordination: when a process may act depends on what the others have done or are doing.

not before

P waits until Q has acted



not at the same time

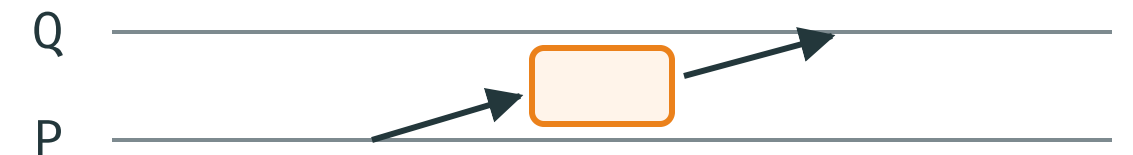
P and Q are never both inside



Communication: data passes from one process to another.

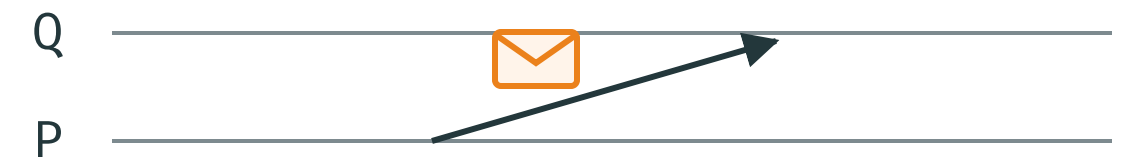
through memory that both can reach

P writes, Q reads



by a message

P sends, Q receives



Coordination and communication in the bounded buffer

// producer

Java ↗

```
for (int k = 1; k <= 8; k++) {  
    String portion = produceNextPortion(k);  
    numberOfEmptyPositions.acquire();  
    bufferManipulation.acquire();  
    buffer.addLast(portion);  
    bufferManipulation.release();  
    numberOfQueuingPortions.release();  
}
```

// consumer

```
for (int k = 1; k <= 8; k++) {  
    numberOfQueuingPortions.acquire();  
    bufferManipulation.acquire();  
    String portion = buffer.removeFirst();  
    bufferManipulation.release();  
    numberOfEmptyPositions.release();  
    processPortionTaken(portion);  
}
```

not before: the consumer takes no portion before one is produced.

`numberOfQueuingPortions` counts the portions in the queue: `acquire` waits while it is 0, the producer's `release` adds one.

not before: the producer adds no portion before there is room.

`numberOfEmptyPositions` counts the empty places: `acquire` waits while it is 0, the consumer's `release` adds one.

not at the same time: the two never change the queue at once.

`bufferManipulation` is taken before the queue is touched and given back after.

communication: a portion passes through the queue, memory that both reach.

`addLast` in the producer, `removeFirst` in the consumer.

Correctness: safety and liveness

“Software is hard.”

Donald Knuth

Writing a correct sequential program is already a challenge. For a concurrent or distributed program it is an **order of magnitude** harder: one input has many executions, and a part can fail while the others go on.

In general, the correctness of a program splits into properties of two kinds.

Safety

Nothing bad ever happens.

A violation happens at one definite moment.

Liveness

Something good eventually happens.

No finite beginning of an execution violates it.

Common correctness properties

property	what it says	an example	
mutual exclusion	two processes are never in the same critical section at the same time	the bounded buffer: one at a time on the queue	safety
linearizability	the calls answer as if made one at a time, in an order that keeps a call that returned before one that was made later	a bank account that many clients use at once	safety
deadlock freedom	no state is ever reached in which each process waits for something that only another can give	two transfers, each holding one lock and waiting for the other's	safety
starvation freedom	every process that asks is served in the end	every request to a server gets an answer	liveness
eventual consistency	if the requests stop, the copies agree in the end	two copies of an account on two machines	liveness

Language support for correctness

For every property, who makes it hold: **the language**, **a library**, or **the programmer**?

	the language (compiler, runtime)	a library	the programmer (keywords, discipline)
mutual exclusion safety	Rust: no way to the data in a <code>Mutex<T></code> without its lock. Erlang: nothing is shared.	locks and semaphores: <code>java.util.concurrent</code>	<code>synchronized</code> , <code>sync.Mutex</code> : every access must lock; nothing checks
starvation freedom liveness	Go, Erlang interrupt a long computation; JavaScript does not	fair locks: <code>new ReentrantLock(true)</code>	no long computation on the loop; time limits

This is not a course on concurrency or on distributed systems: it meets their notions through the programming language.

mutual exclusion	which construct holds the lock, and who checks that it is taken?
failures	in what form does a failure reach the program?

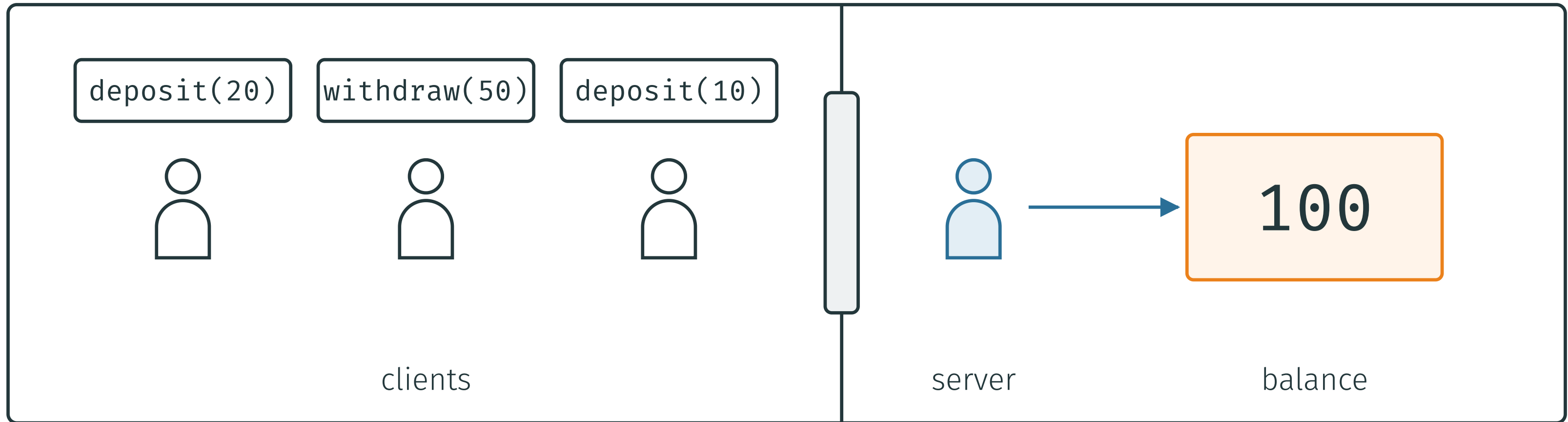
Overview of Part 2

The questions of Part 2

- 6 At which points of a function's text can control pass to another process, and what represents the suspended computation?
- 7 When two processes can reach the same data, which constructs express mutual exclusion and condition synchronization, and do they compose?
- 8 If processes share no data, what does passing a value from one to the other mean: is it shared, copied, or moved?
- 9 When a fault occurs, in what form does it reach the program, which constructs report it and recover from it, and what remains for the programmer?

Every week: who makes each property hold: the language, a library, or the programmer?

A running example: a bank account



`deposit(n)` answers **ok**

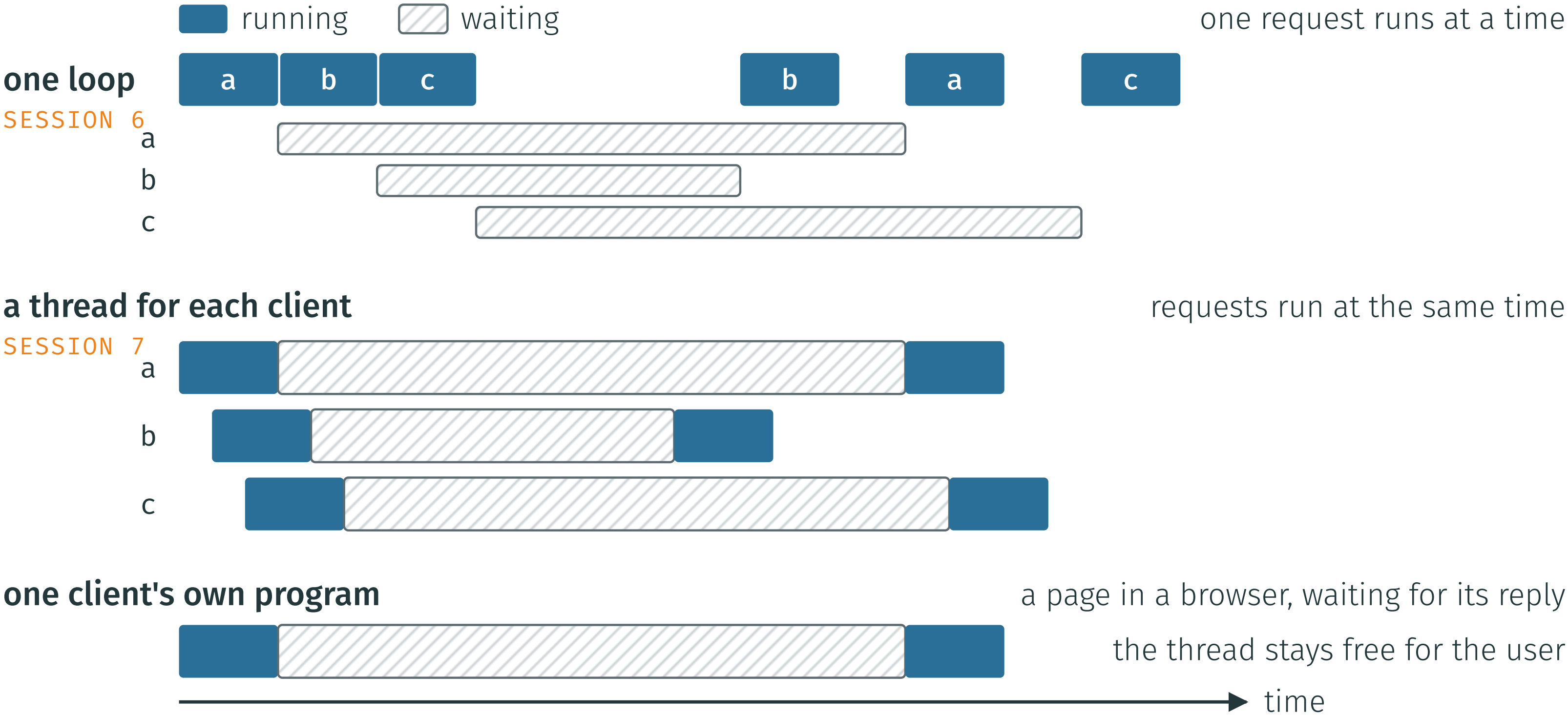
`withdraw(n)` answers **ok** or **refused**

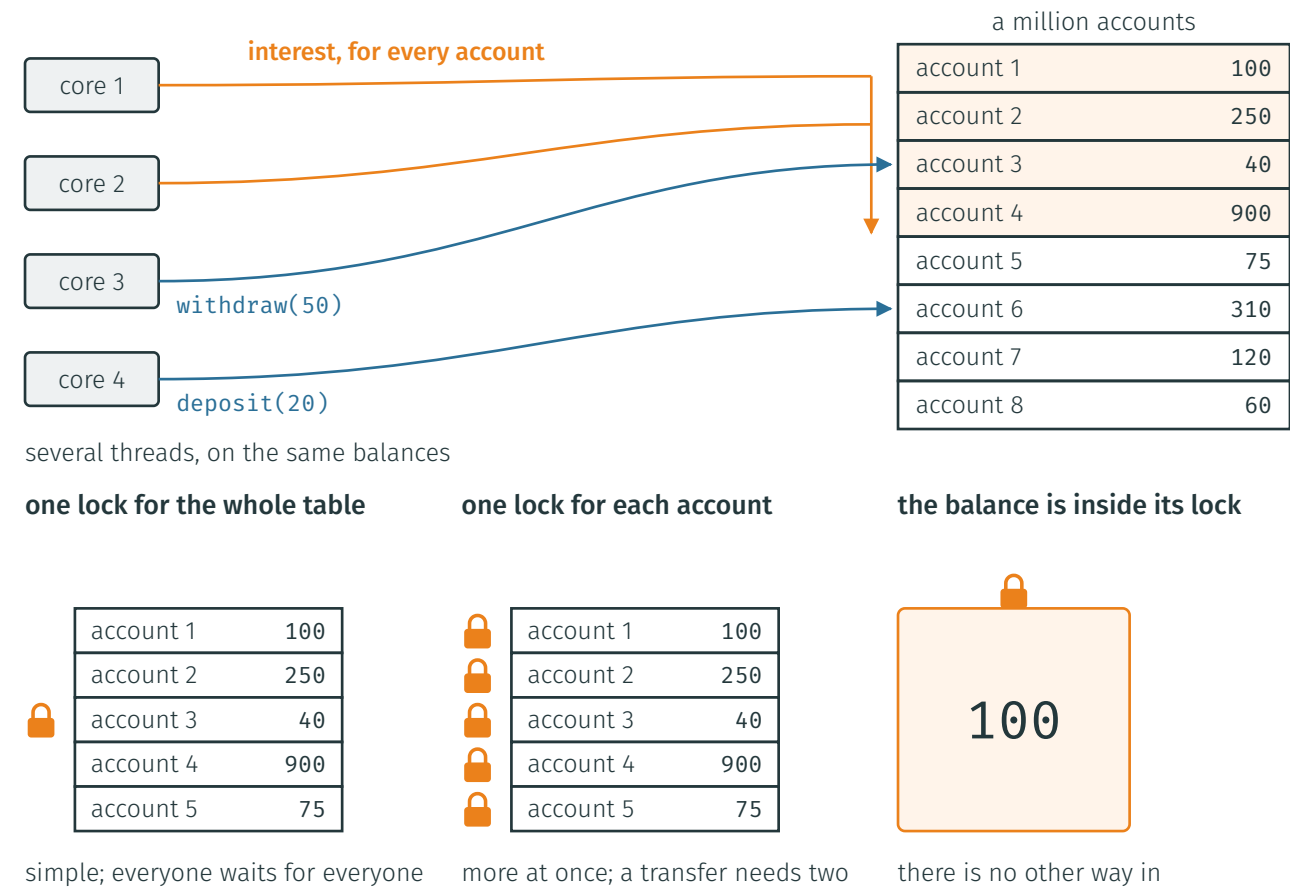
as if one at a time, in some order

From outside, the bank must behave as **one sequential machine**.

Linearizability: in every execution, the calls can be arranged in one sequence in which each call answers as it would if the calls were made alone, in that order, and a call that returned before another was made comes first.

Serving many clients: one loop, or a thread each





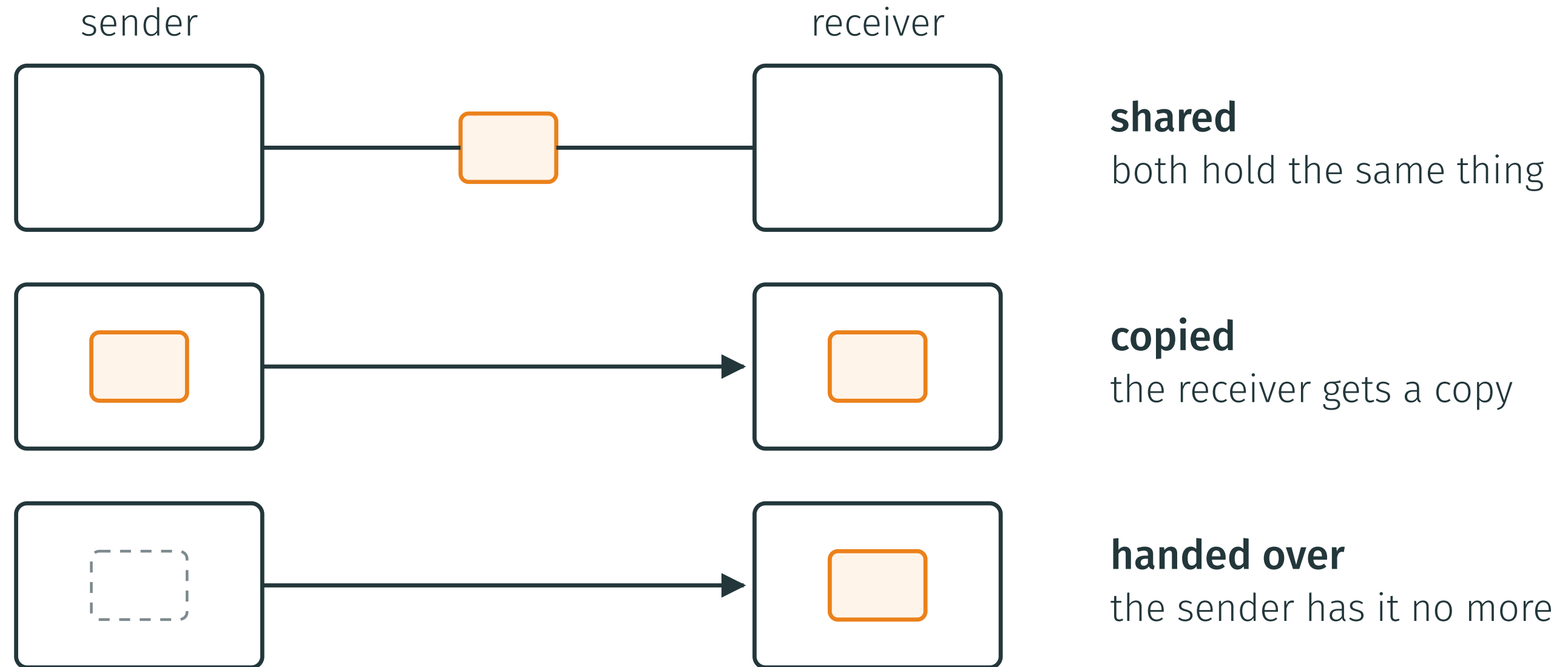
One loop is not enough: the interest on a million accounts would freeze every client, and the machine has several cores. So several threads run at once on the same balances, and two are about to change the same balance at the same time: the **one at a time** of Dijkstra’s queue.

Two processes need the same data: share it, or send copies?

- On different machines there is no memory to share.
- And sharing is hard: threads are “wildly nondeterministic” (Lee, 2006).

Message passing: isolated parts with mailboxes

SESSION 8



shared: nothing is copied, but if the thing can change we are back at locks

copied: the price is the copying · handed over: the language must stop the sender using it afterwards

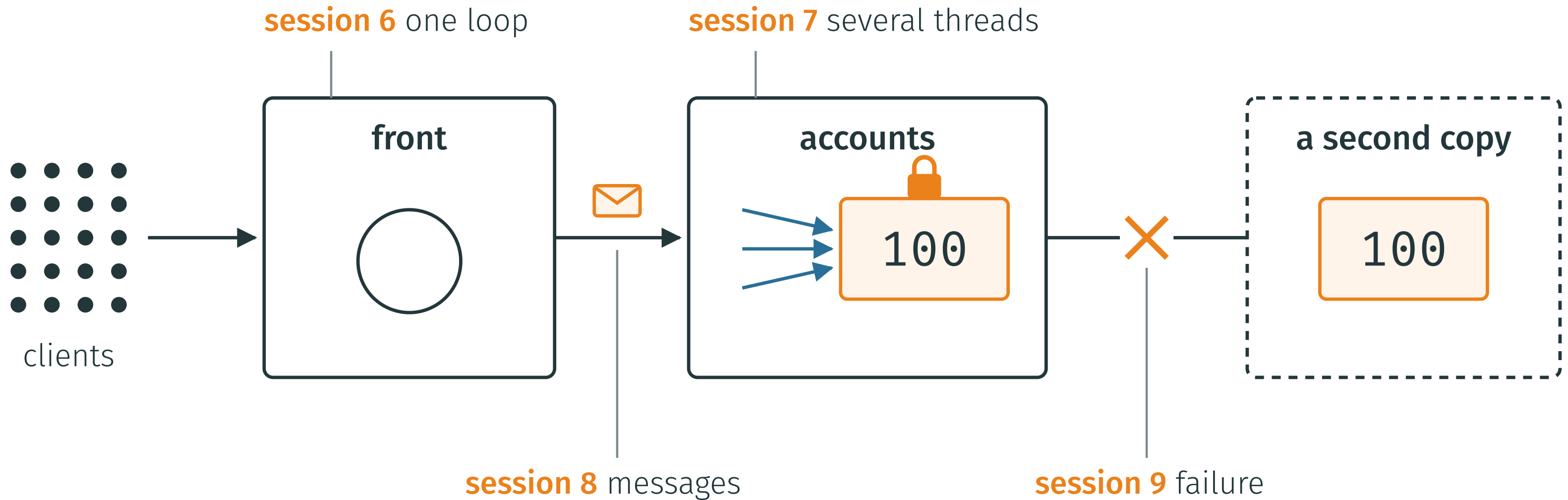
Replication: two copies and a failed link

SESSION 9

Two copies cannot reach each other: what does a copy answer to a client? Refuse to answer, or let them differ?



Part 2 at a glance



What does a program do while it waits, and where is the rest of a function while it waits?

Appendix

The bounded buffer in asynchronous style

Run k produced k processed

run 1	run 3
212 ms	230 ms
1	1
2	2
3	3
1	4
4	1
5	5
2	2
6	6
3	3
7	7
4	4
8	8
5	5
6	6
7	7
8	8

run 3: a different order

Every run prints portions 1 to 8 in order, each after it is produced.

Java ↗

```
// one at a time on the queue: one thread runs all of this
static final ScheduledExecutorService loop =
    Executors.newSingleThreadScheduledExecutor();
// producer
static CompletableFuture<Void> produce(int k) {
    if (k > 8) return done();
    String portion = produceNextPortion(k);
    return queue.put(portion) // wait for room
        .thenComposeAsync(v -> produce(k + 1), loop);
}
// consumer
static CompletableFuture<Void> consume(int k) {
    if (k > 8) return done();
    return queue.take() // wait for a portion
        .thenComposeAsync(p -> processPortionTaken(p), loop)
        .thenComposeAsync(v -> consume(k + 1), loop);
}
```

The animation of the bounded buffer

The diagram of slide 3 is a program: two `async` functions sharing one array as the queue.

`until` waits and `signal` wakes, in the parts of `acquire` and `release`.

The device's time is random: every run differs.

One thread: the turn passes only at an `await`, so one at a time on the queue needs no semaphore. Session 6.

```
// producer
for (let k = 1; k <= 8; k++) {
  await sleep(PRODUCE[(k - 1) % 3]); // produce next portion
  const c = chip(k);
  while (queue.length === 2)        // wait for room
    await until('room', producer);
  queue.push(c); place();           // add portion to buffer
  signal('portion');
}

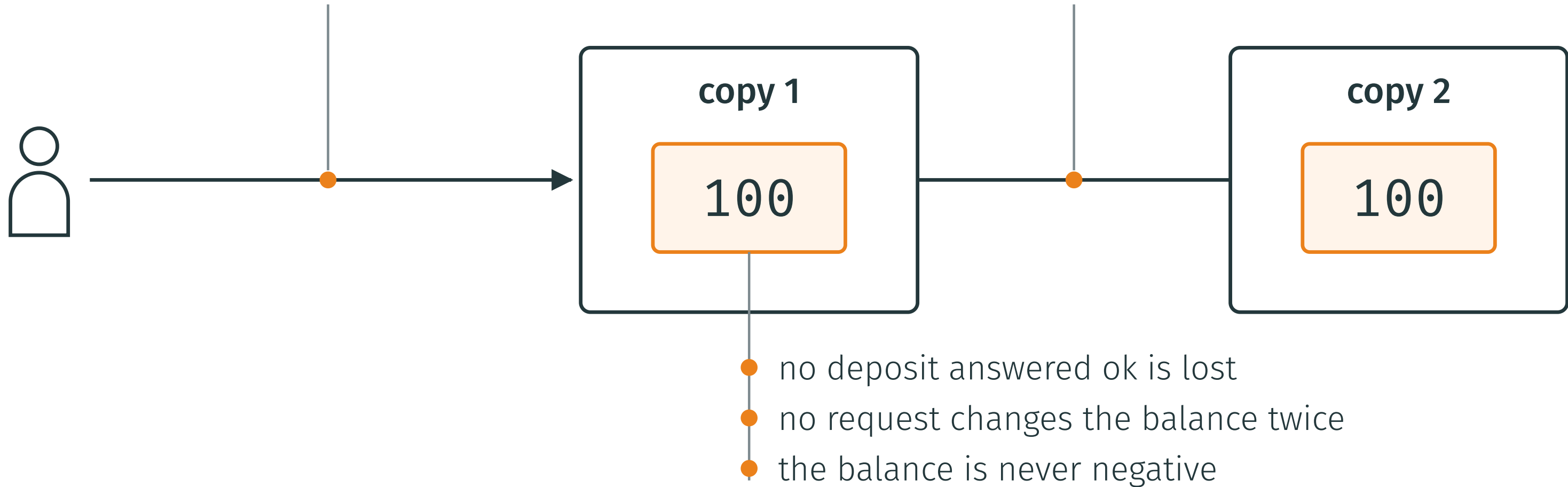
// consumer
for (let k = 1; k <= 8; k++) {
  while (queue.length === 0)        // wait for a portion
    await until('portion', consumer);
  const c = queue.shift(); place(); // take portion from buffer
  signal('room');
  moveTo(c, AT_CONSUMER);
  await sleep(0.8 + Math.random() * 0.9); // the device
  print(c, k);                          // process portion taken
}
```

JavaScript ↗

Example: the requirements of a bank account

every request gets a definitive answer in the end

the copies agree



Liveness: the request gets its answer in the end; the copies agree in the end.

Safety: the three requirements at the balance. Each says *never*.

Language support, property by property

	the language (compiler, runtime)	a library	the programmer (keywords, discipline)
mutual exclusion	Rust: no way to the data in a <code>Mutex<T></code> without its lock. Erlang: nothing is shared.	locks and semaphores: <code>java.util.concurrent</code>	<code>synchronized</code> , <code>sync.Mutex</code> : every access must lock; nothing checks
linearizability	Erlang: a process handles one message at a time	<code>AtomicInteger</code> : each operation, alone	two such operations in a row are not one: check, then act
deadlock freedom	Prevented by none of Java, Rust, Go, Erlang. Go's runtime detects a total deadlock.	<code>tryLock</code> with a time limit	one order in which all code takes its locks
starvation freedom	Go, Erlang interrupt a long computation; JavaScript does not	fair locks: <code>new ReentrantLock(true)</code>	no long computation on the loop; time limits
eventual consistency	Provided by no general-purpose language.	mergeable data types, such as a set with <code>union</code>	which merge is right; when to send again

Session 6: Asynchronous Programming

concurrency on one thread

- One thread runs all our code. A function that waits gives the thread back.
- We look at what a language does at an `await`.

After this session we can explain:

- where the rest of a function is kept while it waits;
- why waiting is not running, and why one thread does not rule out a race.

Languages: JavaScript; Python beside it.

```
function drive(gen) {  
  const g = gen();  
  const step = v => {  
    const r = g.next(v);  
    if (!r.done) r.value.then(step);  
  };  
  step();  
}
```

JavaScript ↗

live
A
B
C Ada

Run

```
async def main():  
    asyncio.create_task(load())  
    print("B")  
    await asyncio.sleep(1)
```

Python ↗

live
B
A
C Ada

Run

Session 7: Multi-Threading

shared-memory concurrency: locks, atomic operations, ownership

- Several threads run our code at the same time, on the same memory.
- We look at how shared data is protected, who checks that it is, and when nothing needs protecting.

After this session we can explain:

- why a lock protects only if every access takes it, and how a compiler can check that;
- why two protected operations, one after the other, are not one protected operation;
- why a parallel `map` of a function without effects gives the same result in every execution.

Languages: Java; Rust and OCaml beside it.

```
let balance = Arc::new(Mutex::new(100));  
let b = Arc::clone(&balance);  
let t = thread::spawn(move || {  
    *b.lock().unwrap() += 20;  
});  
t.join().unwrap();
```

Rust ↗

recorded · Rust
120

Run

```
long total = IntStream.rangeClosed(1, N)  
    .parallel()  
    .mapToLong(Sum::f)  
    .sum();
```

Java ↗

recorded · Java 26
sequential 1999999
parallel 1999999

Run

Session 8: Message Passing

concurrency without shared memory: channels and actors

- Processes share no data. They send each other messages, and every message sent arrives.
- We look at what is addressed (a channel, or a process), and at what sending does to a value.

After this session we can explain:

- why a process that handles one message at a time needs no lock on its own state;
- what happens to a value that is sent: shared, copied, or handed over;
- why messages remove neither the deadlock nor the lost deposit.

Languages: Go; Erlang.

```
portions := make(chan string, 2)
go func() {
    for k := 1; k <= 8; k++ {
        portions <- produce(k)
    }
    close(portions)
}()
for p := range portions {
    process(p)
}
```

Go ↗

recorded · Go 1.27

line 1
line 2
line 3
line 4
line 5
line 6
line 7
line 8

Run

```
loop(Balance) ->
    receive
        {deposit, N, From} ->
            From ! ok, loop(Balance + N);
        {withdraw, N, From} when N <= Balance ->
            From ! ok, loop(Balance - N);
        {withdraw, _, From} ->
            From ! refused, loop(Balance)
    end.
```

Erlang ↗

Run

recorded · Erlang ok refused ok

Session 9: Failure and Recovery

distributed programming: delivery not assumed

- A message can be lost, a process or a machine can stop, and the rest goes on.
- We look at the form in which a failure reaches a program, and at what the program can do then.

After this session we can explain:

- why a process that is slow and a process that has stopped look the same from outside;
- when a request may be sent again;
- what a restart gives back, and what it does not.

Languages: Erlang; OCaml.

```
ask(Account, Request) ->  
  Account ! Request,  
  receive  
    Answer -> Answer  
  after 1000 ->  
    no_answer  
end.
```

Erlang ↗

recorded · Erlang
no_answer

Run

```
module Entries = Set.Make (String)  
let merge a b = Entries.union a b
```

OCaml ↗

Run

```
recorded · OCaml 5.4  
merge b a = merge a b: true  
merge a a = a: true
```